Large Project Decomposition

ECE 2564, Embedded Systems

Fall 2025

Overall Code Structure of a Large Project

Game: Guess the Color

We'll review the finite state machine (FSM) and the program

Code: **Import this repository**
Videos: **Watch these videos**

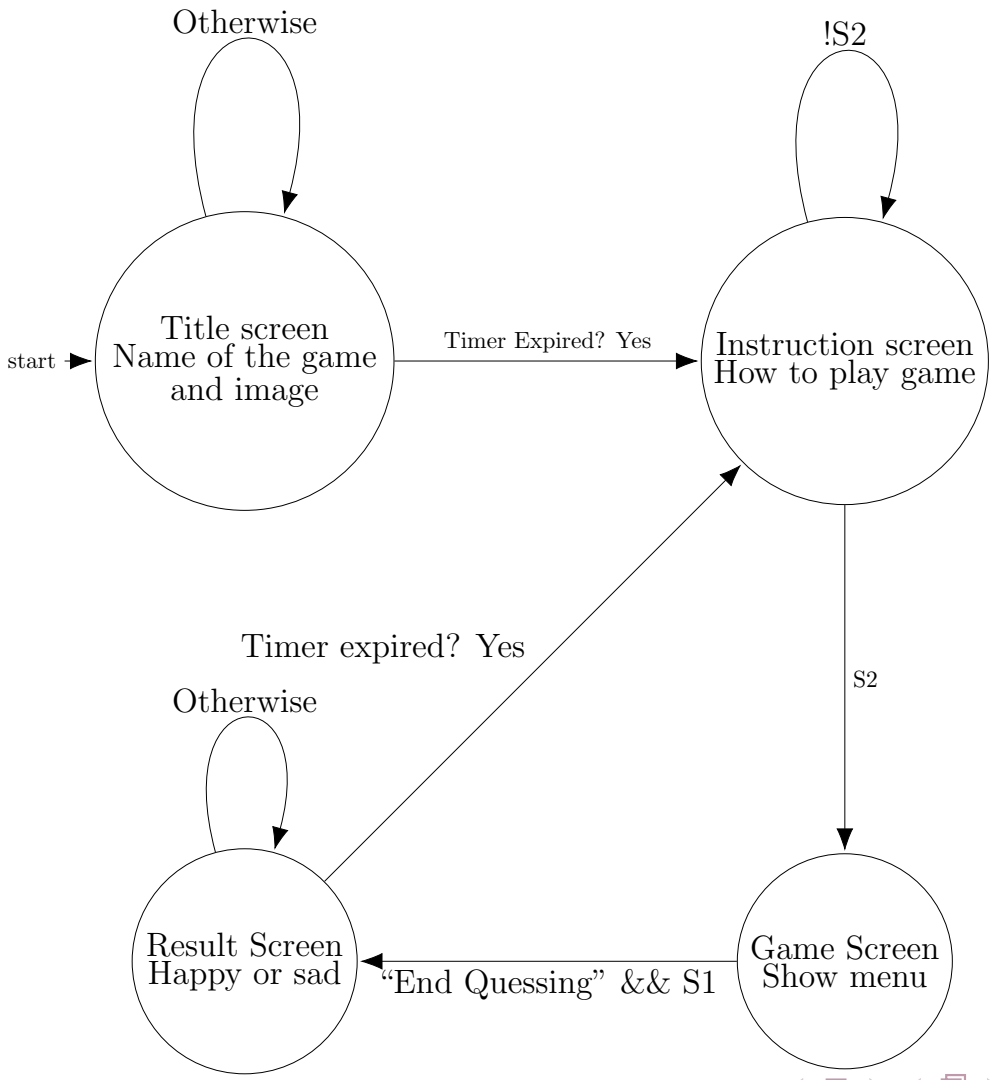
Let's Play the Game

- Compile it and flash it to MSP432
- Watch the LCD on the BoosterPack and follow the instructions shown on the display
- Use the buttons B1 and B2 on the BoosterPack to play the game

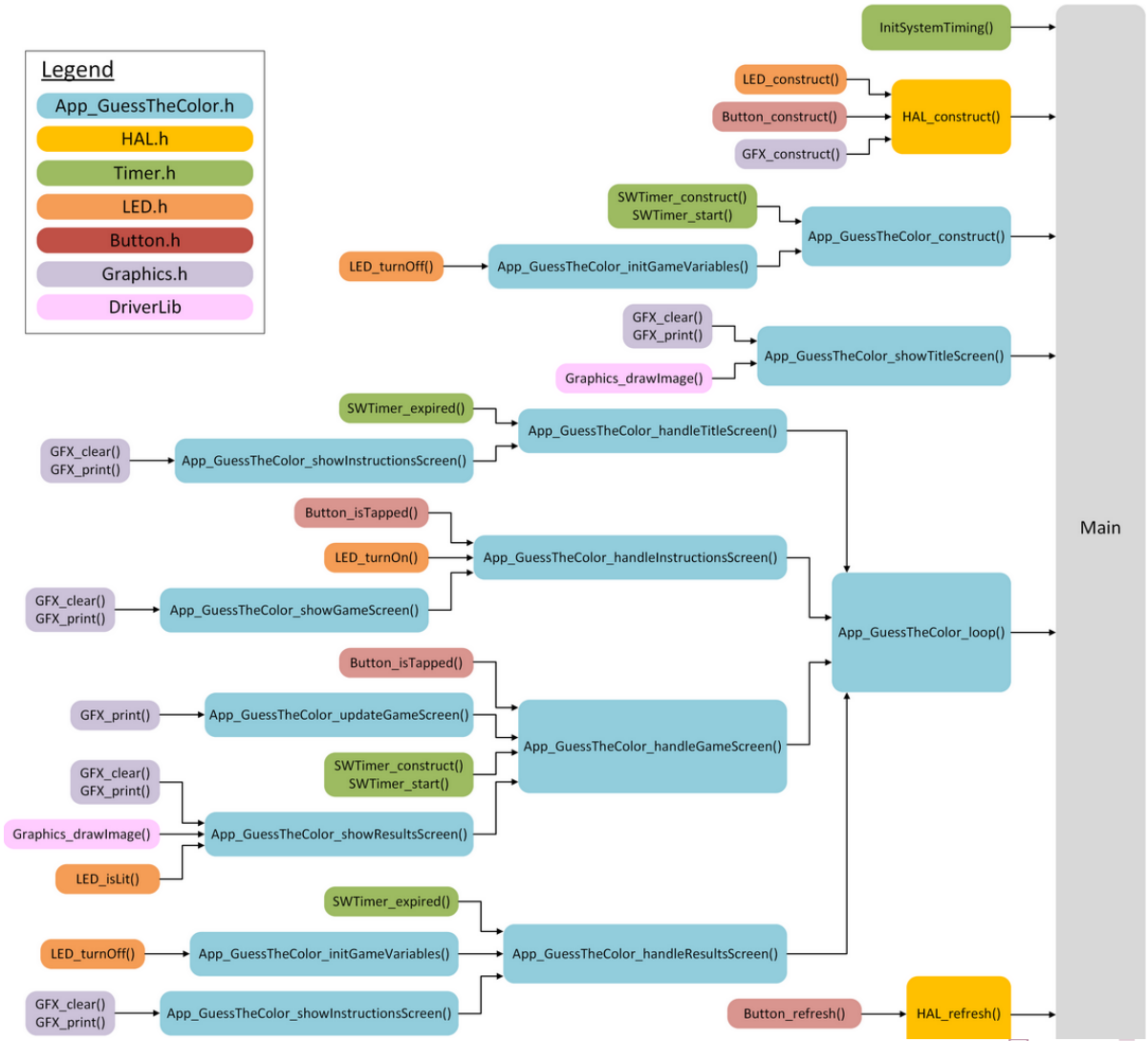
Motivation

- Large project management
- How to program it

The Finite State Machine (FSM)



Code Structure



High Level Code Structure

- HAL: display, buttons and LED
- Application: game control

```
int main(void)
{
    // Stop Watchdog Timer - THIS SHOULD ALWAYS BE THE FIRST LINE OF YOUR MAIN
    WDT_A_holdTimer();

    // Initialize the system clock and background hardware timer, used to enable
    // software timers to time their measurements properly.
    InitSystemTiming();

    // Initialize the main Application object and the HAL.
    HAL hal = HAL_construct();
    App_GuessTheColor app = App_GuessTheColor_construct(&hal);
    App_GuessTheColor_showTitleScreen(&hal.gfx);

    // Main super-loop! In a polling architecture, this function should call
    // your main FSM function over and over.
    while (true)
    {
        App_GuessTheColor_loop(&app, &hal);  //update my program, application state, output
        HAL_refresh(&hal); // check the inputs
    }
}
```

Constants and Enums: App_GuessTheColor.h

- GameState: state of the FSM
- Cursor: the selection made by the player

```
#define TITLE_SCREEN_WAIT   3000  //3 seconds
#define RESULT_SCREEN_WAIT  3000  // 3 seconds

#define NUM_TEST_OPTIONS    4
#define NUM_RANDOM_NUMBERS  5

enum _GameState
{
    TITLE_SCREEN, INSTRUCTIONS_SCREEN, GAME_SCREEN, RESULT_SCREEN
};
typedef enum _GameState GameState;

enum _Cursor
{
    CURSOR_0 = 0, CURSOR_1 = 1, CURSOR_2 = 2, CURSOR_3 = 3
};
typedef enum _Cursor Cursor;
```

High-Level Application Loop: App_GuessTheColor.c

```
/**
 * The main super-loop function of the application. We place this inside of a
 * single infinite loop in main. In this way, we can model a polling system of
 * FSMs. Every cycle of this loop function, we poll each of the FSMs one time.
 */
void App_GuessTheColor_loop(App_GuessTheColor* app_p, HAL* hal_p)
{
    switch (app_p->state)
    {
        case TITLE_SCREEN:
            App_GuessTheColor_handleTitleScreen(app_p, hal_p);
            break;
        case INSTRUCTIONS_SCREEN:
            App_GuessTheColor_handleInstructionsScreen(app_p, hal_p);
            break;
        case GAME_SCREEN:
            App_GuessTheColor_handleGameScreen(app_p, hal_p);
            break;
        case RESULT_SCREEN:
            App_GuessTheColor_handleResultScreen(app_p, hal_p);
            break;
        default:
            break;
    }
}
```

Title Screen: App_GuessTheColor.c

```
/**
 * A helper function which clears the screen and prints the instructions for how
 * to play the game.
 */
void App_GuessTheColor_showInstructionsScreen(App_GuessTheColor* app_p, GFX* gfx_p)
{
    // Clear the screen from any old text state
    GFX_clear(gfx_p);

    // Display the text
    GFX_print(gfx_p, "Instructions          ", 0, 0);
    GFX_print(gfx_p, "----------------------", 1, 0);
    GFX_print(gfx_p, "Guess the RGB mix.    ", 2, 0);
    GFX_print(gfx_p, "Controls during guessing:", 3, 0);

    GFX_print(gfx_p, "B1: Select choice     ", 5, 0);
    GFX_print(gfx_p, "B2: Move arrow        ", 6, 0);

    GFX_print(gfx_p, "Press B2 to start.    ", 8, 0);
}
```

Generating a Random Color: App_GuessTheColor.c

- Pre-populate an array of random numbers
- Why focus on the least 3 significant bits of random number?

```
/**
 * Callback function for when the game is in the INSTRUCTIONS_SCREEN state. Used
 * to break down the main App_GuessTheColor_loop() function into smaller
 * sub-functions.
 */
void App_GuessTheColor_handleInstructionsScreen(App_GuessTheColor* app_p, HAL* hal_p)
{
    // Transition to start the game when B2 is pressed
    if (Button_isTapped(&hal_p->boosterpackS2))
    {
        // Update internal logical state
        app_p->state = GAME_SCREEN;

        // Turn on LEDs based off of the lowest three bits of a random number.
        uint32_t randomNumber = app_p->randomNumbers[app_p->randomNumberChoice];

        if (randomNumber & BIT0) { LED_turnOn(&hal_p->boosterpackRed  ); }
        if (randomNumber & BIT1) { LED_turnOn(&hal_p->boosterpackGreen); }
        if (randomNumber & BIT2) { LED_turnOn(&hal_p->boosterpackBlue ); }

        // Increment the random number choice with a mod loopback to 0 when reaching
        // NUM_RANDOM_NUMBERS.
        app_p->randomNumberChoice = (app_p->randomNumberChoice + 1) % NUM_RANDOM_NUMBERS;

        // Display the next state's screen to the user
        App_GuessTheColor_showGameScreen(app_p, &hal_p->gfx);
    }
}
```

Playing the Game: App_GuessTheColor.c

```
void App_GuessTheColor_handleGameScreen(App_GuessTheColor* app_p, HAL* hal_p)
{
    // If B2 is pressed, increment the cursor and circle it around to 0 if it
    // reaches the bottom
    if (Button_isTapped(&hal_p->boosterpackS2)) {
        app_p->cursor = (Cursor) (((int) app_p->cursor + 1) % NUM_TEST_OPTIONS);
        App_GuessTheColor_updateGameScreen(app_p, &hal_p->gfx);
    }

    // If B1 is pressed, either add a selection to the proper color choice OR
    // transition to the SHOW_RESULT state if the user chooses to end the test.
    if (Button_isTapped(&hal_p->boosterpackS1))
    {
        switch (app_p->cursor)
        {
            // In the first three choices, we need to re-display the game screen
            // to reflect updated choices.
            // ---------------------------------------------------------------
            case CURSOR_0: // Red choice
                app_p->redSelected = true;
                App_GuessTheColor_updateGameScreen(app_p, &hal_p->gfx);
                break;

            case CURSOR_1: // Green choice
                app_p->greenSelected = true;
                App_GuessTheColor_updateGameScreen(app_p, &hal_p->gfx);
                break;

            case CURSOR_2: // Blue choice
                app_p->blueSelected = true;
                App_GuessTheColor_updateGameScreen(app_p, &hal_p->gfx);
                break;
```

Playing the Game: App_GuessTheColor.c

```
            // In the final choice, we must setup a transition to RESULT_SCREEN
            // by starting a timer and calling the proper draw function.
            // ------------------------------------------------------------------
            case CURSOR_3:
                app_p->state = RESULT_SCREEN;

                app_p->timer = SWTimer_construct(RESULT_SCREEN_WAIT);
                SWTimer_start(&app_p->timer);

                App_GuessTheColor_showResultScreen(app_p, hal_p);
                break;

            default:
                break;
        }
    }
}
```

Updating the Screen During Game Play: App_GuessTheColor.c

- Why clear the arrow to the choices first?
  - No need to remember where the arrow was
  - You don't have to do it this way
- Draw asterisks to indicate the choices

```
/**
 * A helper function which updates the main game screen by redrawing only the
 * positions where the cursor could possibly be updated.
 */
void App_GuessTheColor_updateGameScreen(App_GuessTheColor* app_p, GFX* gfx_p)
{
    // Clear the cursors from any previous game screen
    GFX_print(gfx_p, "  ", 2, 0);
    GFX_print(gfx_p, "  ", 3, 0);
    GFX_print(gfx_p, "  ", 4, 0);
    GFX_print(gfx_p, "  ", 5, 0);

    // Draw the cursor
    GFX_print(gfx_p, ">", 2 + app_p->cursor, 0);

    // Draw the stars for LED guesses
    if (app_p->redSelected  ) { GFX_print(gfx_p, "*", 2, 8); }
    if (app_p->greenSelected) { GFX_print(gfx_p, "*", 3, 8); }
    if (app_p->blueSelected ) { GFX_print(gfx_p, "*", 4, 8); }
}
```

Results – Winning or Losing: App_GuessTheColor.c

- How do we check if the guess is correct?

```
/**
 * A helper function which clears the screen and displays whether the user has
 * won or not.
 */
void App_GuessTheColor_showResultScreen(App_GuessTheColor* app_p, HAL* hal_p)
{
    // Print the splash text
    GFX_clear(&hal_p->gfx);
    GFX_print(&hal_p->gfx, "Result                ", 0, 0);
    GFX_print(&hal_p->gfx, "--------------------", 1, 0);

    // Determine if each selection matched correctly
    bool match = app_p->redSelected   == LED_isLit(&hal_p->boosterpackRed  )
              && app_p->greenSelected == LED_isLit(&hal_p->boosterpackGreen)
              && app_p->blueSelected  == LED_isLit(&hal_p->boosterpackBlue );

    // Print the correct string based on if the user won or not
    if (match) {
        GFX_print(&hal_p->gfx, "Right!", 2, 0);
        Graphics_drawImage((const Graphics_Context *)&hal_p->gfx, &happy8BPP_UNCOMP, 60, 60);
    }
    else {
        GFX_print(&hal_p->gfx, "Wrong!", 2, 0);
        Graphics_drawImage((const Graphics_Context *)&hal_p->gfx, &sad8BPP_UNCOMP, 60, 60);
    }
}
```

Starting a New Game After Win/Lose: App_GuessTheColor.c

- Clear all variables
- Turn of all LEDs

```
/**
 * A helper function which resets all the game variables to their unselected
 * states and resets the cursor position.
 */
void App_GuessTheColor_initGameVariables(App_GuessTheColor* app_p, HAL* hal_p)
{
    // Reset the cursor
    app_p->cursor = CURSOR_0;

    // Deselect each option
    app_p->redSelected = false;
    app_p->greenSelected = false;
    app_p->blueSelected = false;

    // Turn off all LEDs - they don't turn on until a random number is generated
    LED_turnOff(&hal_p->boosterpackRed);
    LED_turnOff(&hal_p->boosterpackGreen);
    LED_turnOff(&hal_p->boosterpackBlue);
}
```

Summary

- To create a new software pront don't start by writing code
- THINK: *top-down design; botton-up implementation*

- On paper, subdivide the project into function parts
- Consider using paper and pencil to speify a high-level FSM
- Decide how to partition the code using ...
  - Higher-level functions that you may need to specify
  - HAL functions that you may need to specify
  - DriverLib functions provided by TI
- Write the code
  - Set up some starter code if not provided to you in which many of the function are simple placeholders
  - Write and test separate parts of the code individually
  - Do not try to write all of the code and then try to debug